

Document Number: **P0670R3**, ISO/IEC JTC1 SC22 WG21
Audience: CWG, LWG
Date: 2018-05-07
Authors: Matúš Chochlík (chochlik@gmail.com)
Axel Naumann (axel@cern.ch)
David Sankel (camior@gmail.com)

Function reflection

0. Introduction

[P0194](#) introduced static reflection for types and variables. This paper adds static reflection of functions.

Reflection proposed here behaves as follows:

The functionality introduced here allows for reflection of calls of concrete functions. This enables, for instance, GUI generation, building a catalogue for remote procedure calls, documentation generation, and signal/slot frameworks. Like P0194, this proposal omits attributes, templates, and reification (i.e. conversion of a meta object to the base level): all warrant a separate paper. We would welcome a paper especially on static reflection of attributes, matching the interface-style of P0194 and this paper! Linkage and friends will be part of a follow-up paper to P0194; they will have a combined "effect" on P0194 and this paper.

0.1 Interplay with other proposals

Most notably, this proposal relies on the Reflection TS and the Concepts TS.

[P0385](#) discusses use cases, rationale, design decisions, and the future evolution of the proposed reflection facility. It also has usage examples and replies to frequently asked questions.

1 Scope

[intro.scope]

This document is written as a set of changes against the Reflection TS ([N4746](#)). Instructions to modify or add paragraphs are written as explicit instructions. Modifications made directly to existing text from the Reflection TS use underlining to represent added text and ~~striketrough~~ to represent deleted text.

2 Normative references

[intro.refs]

No changes are made to Clause 2 of the Reflection TS.

3 Terms and definitions

[intro.defs]

No changes are made to Clause 3 of the Reflection TS.

4 General

[intro]

4.3 Acknowledgements

[intro.ack]

Modify the section as follows:

This work is the result of a collaboration of researchers in industry and academia. We wish to thank people who made valuable contributions within and outside these groups, including Ricardo Fabiano de Andrade, Roland Bock, Chandler Carruth, [Jackie Kay](#), Klaim-Joël Lamotte, Jens Maurer, and many others not named here who contributed to the discussion.

5 Lexical conventions [lex]

No changes are made to Clause 5 of the Reflection TS.

6 Basic concepts [basic]

No changes are made to Clause 6 of the Reflection TS.

7 Standard conversions [conv]

No changes are made to Clause 7 of the Reflection TS.

8 Expressions [expr]

No changes are made to Clause 8 of the Reflection TS.

9 Statements [stmt.stmt]

No changes are made to Clause 9 of the Reflection TS.

10 Declarations [dcl.dcl]

10.1 Specifiers [dcl.spec]

10.1.7 Type specifiers [dcl.type]

10.1.7.2 Simple type specifiers [dcl.type.simple]

In C++ [dcl.type.simple], apply the following change

```
...
 $reflexpr\text{-}operand$ :

 $type\text{-}id$ 
 $nested\text{-}name\text{-}specifier_{opt} identifier$ 
 $nested\text{-}name\text{-}specifier_{opt} simple\text{-}template\text{-}id$ 
function call expression
```

10.1.7.6 Reflection type specifier [dcl.type.reflexpr]

Apply the following modification to the enumeration:

- is not the global namespace and is an enclosing namespace of ~~, or~~
- [is a lambda capture of the closure type](#) ,
- [is the closure type of the lambda capture](#) ,
- [is the return type, parameter type, or function type of the function](#) , or
- is reflection-related to an entity or alias and is reflection-related to .

For the operand , the type specified by the satisfies . For an operand that can be parsed as a function call expression [expr.call], the type satisfies . For an operand of the form *identifier* where *identifier* is a template type-parameter, the type satisfies both and .

Modify Table 12 as follows:

Category	<i>identifier</i> or <i>simple-template-id</i> kind	Concept
type	<i>class-name</i> designating a union	
	<i>class-name</i> designating a closure type	
	<i>class-name</i> designating a non-union class	
	<i>enum-name</i>	
	<i>type-name</i> introduced by a <i>using-declaration</i>	both and
	any other <i>typedef-name</i>	both and
namespace	<i>namespace-alias</i>	both and
	any other <i>namespace-name</i>	both and
data member	the name of a data member	
value	the name of a variable or structured binding that is not a local entity	
	the name of an enumerator	both and
	the name of a function parameter	
	the name of a captured entity [expr.prim.lambda.capture]	

Modify the following paragraph as follows:

If the *reflexpr-operand* designates an entity or alias at block scope (6.3.3) or function prototype scope (6.3.4) and the entity is not captured or a function parameter, the program is ill-formed. If the *reflexpr-operand* designates a class member, the type represented by the *reflexpr-specifier* also satisfies . If the *reflexpr-operand* designates a variable or a data member, a function parameter, or a captured entity, it is an unevaluated operand (expr.context). If the *reflexpr-operand* designates both an alias and a class name, the type represented by the *reflexpr-specifier* reflects the alias and satisfies . If the overload resolution of the function call expression is ambiguous, the program is ill-formed.

11 Declarators [dcl.decl]

No changes are made to Clause 11 of the Reflection TS.

13 Derived classes [class.derived]

No changes are made to Clause 13 of the Reflection TS.

14 Member access control [class.access]

No changes are made to Clause 14 of the Reflection TS.

15 Special member functions [special]

No changes are made to Clause 15 of the Reflection TS.

16 Overloading

[over]

No changes are made to Clause 16 of the Reflection TS.

17 Templates

[temp]

No changes are made to Clause 17 of the Reflection TS.

18 Exception handling

[except]

No changes are made to Clause 18 of the Reflection TS.

19 Preprocessing directives

[cpp]

No changes are made to Clause 19 of the Reflection TS.

20 Library introduction

[library]

No changes are made to Clause 20 of the Reflection TS.

21 Language support library

[language.support]

21.11 Static reflection

[reflect]

21.11.1 In general

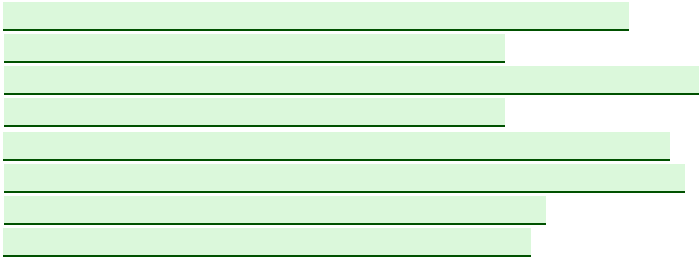
[reflect.general]

21.11.2 Header

synopsis

[reflect.synopsis]

Modify this section as follows:



...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

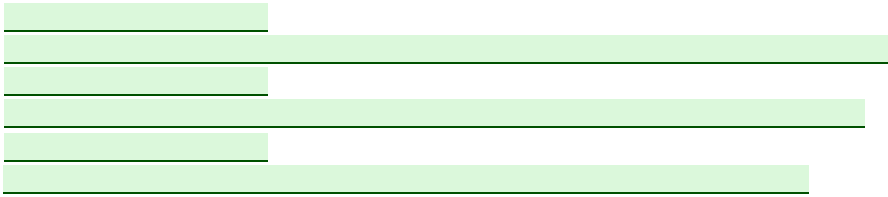
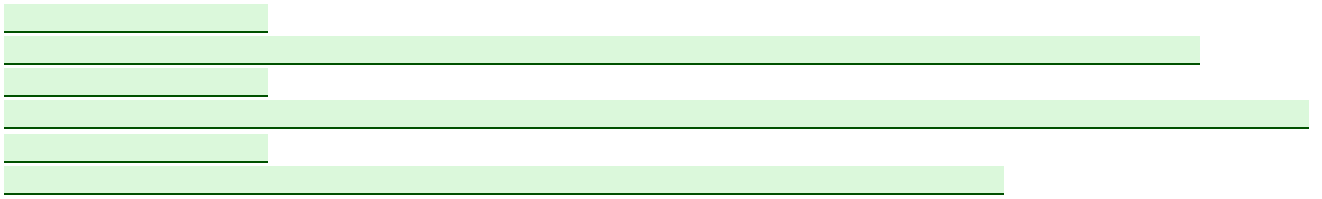
...

...

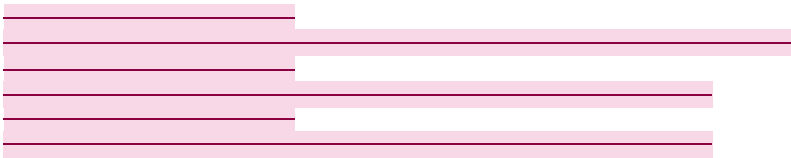
...

...

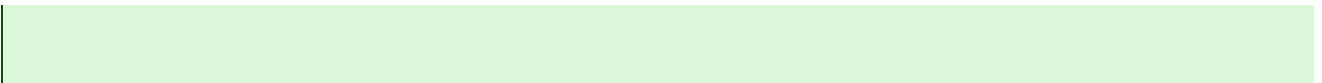
...

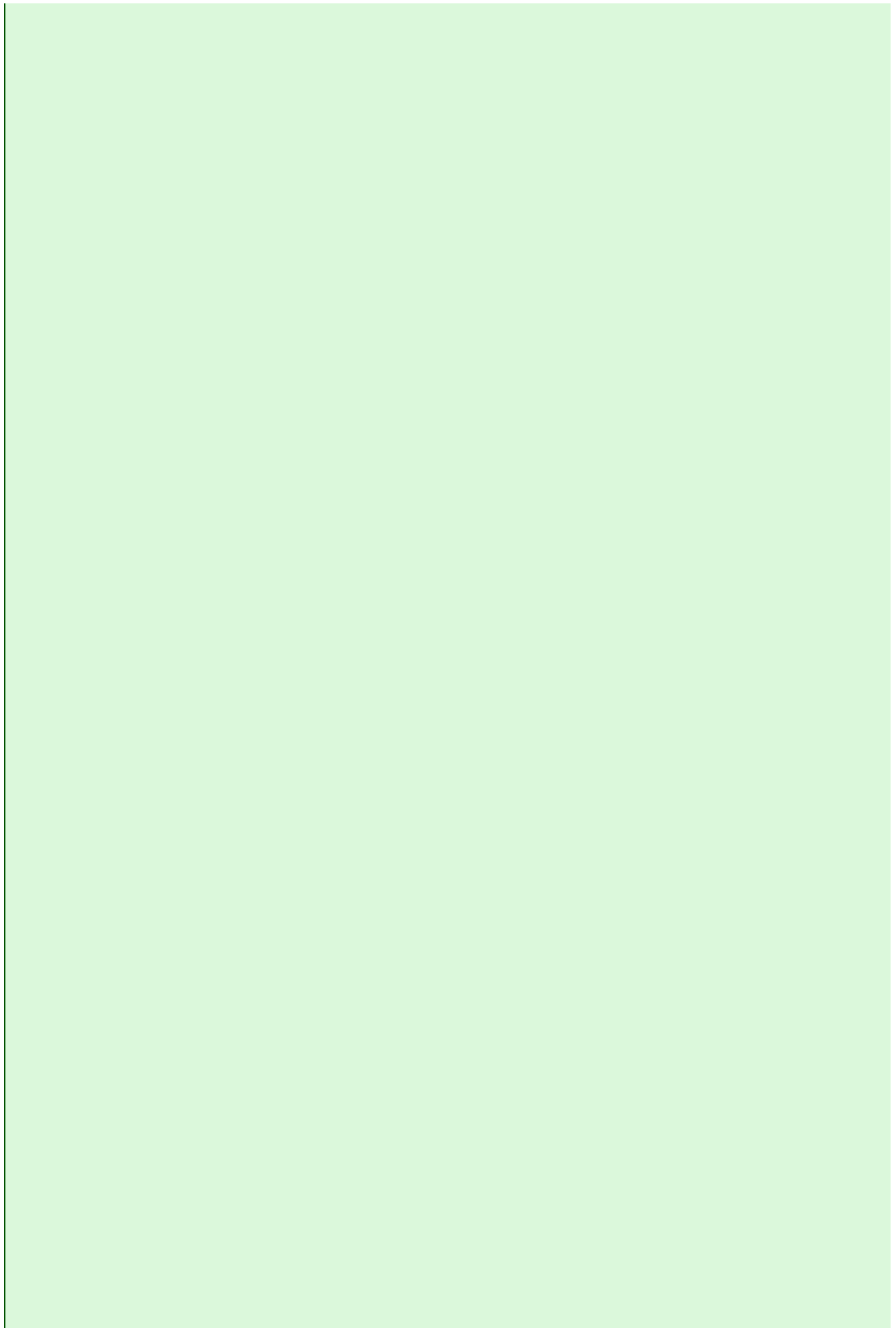


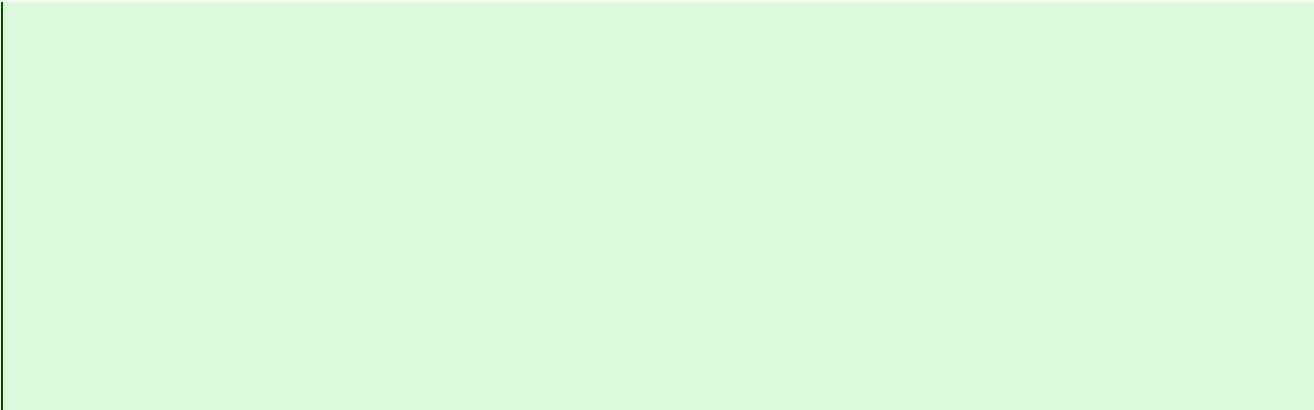
...



...







21.11.3 Concepts for meta-object types

[reflect.concepts]

Add the following paragraphs at the end of [reflect.concepts]:

21.11.3.19 Concept	[reflect.concepts.fctparam]
is satisfied if and only if reflects a function patameter. Any such also satisfies and . The of a is the this parameter appertains to.	
21.11.3.20 Concept	[reflect.concepts.callable]
is satisfied if and only if reflects a function including operators, constructors and destructors, or an object with a call operator, for instance a closure type. Any such also satisfies , and .	
21.11.3.21 Concept	[reflect.concepts.expr.fctcall]
is satisfied if and only if reflects a function call expression [expr.call]. Any such also satisfies .	
21.11.3.22 Concept	[reflect.concepts.fct]
is satisfied if and only if reflects an object with a call operator (for instance a closure object) or a function, excluding constructors and destructors. Any such also satisfies and .	
21.11.3.23 Concept	[reflect.concepts.memfct]

is satisfied if and only if `reflect.isMemberFunction` reflects a member function, excluding constructors and destructors. Any such `reflect.isMemberFunction` also satisfies `reflect.isFunction` and `reflect.isCallable`.

21.11.3.24 Concept `reflect.concepts.specialfct` **[reflect.concepts.specialfct]**

is satisfied if and only if `reflect.isSpecialMemberFunction` reflects a special member function [special]. Any such `reflect.isSpecialMemberFunction` also satisfies `reflect.isMemberFunction`.

21.11.3.25 Concept `reflect.concepts.ctor` **[reflect.concepts.ctor]**

is satisfied if and only if `reflect.isConstructor` reflects a constructor. Any such `reflect.isConstructor` also satisfies `reflect.isMemberFunction` and `reflect.isFunction`. Some specializations of `reflect.isConstructor` satisfy `reflect.isSpecialMemberFunction`.

21.11.3.26 Concept `reflect.concepts.dtor` **[reflect.concepts.dtor]**

is satisfied if and only if `reflect.isDestructor` reflects a destructor. Any such `reflect.isDestructor` also satisfies `reflect.isMemberFunction`, `reflect.isFunction`, and `reflect.isCallable`.

21.11.3.27 Concept `reflect.concepts.oper` **[reflect.concepts.oper]**

is satisfied if and only if `reflect.isOperatorFunction` reflects an operator function [over.oper] or a conversion function [class.conv.fct]. Any such `reflect.isOperatorFunction` also satisfies `reflect.isMemberFunction`. Some specializations of `reflect.isOperatorFunction` satisfy `reflect.isSpecialMemberFunction` or `reflect.isDestructor`.

21.11.3.28 Concept `reflect.concepts.convfct` **[reflect.concepts.convfct]**

is satisfied if and only if `reflect.isConversionFunction` reflects a conversion function [class.conv.fct]. Any such `reflect.isConversionFunction` also satisfies `reflect.isMemberFunction`. Some specializations of `reflect.isConversionFunction` satisfy `reflect.isSpecialMemberFunction`.

21.11.3.29 Concept `reflect.concepts.lambda` **[reflect.concepts.lambda]**

is satisfied if and only if `reflect.isClosureObject` reflects an closure object (excluding generic lambdas). Any such `reflect.isClosureObject` also satisfies `reflect.isObject`.

21.11.3.30 Concept `reflect.concepts.lambdacapture` **[reflect.concepts.lambdacapture]**

is satisfied if and only if reflects a lambda capture as introduced by the capture list or by capture defaults. Any such also satisfies . The of a is its .

21.11.4 Meta-object Operations

[reflect.ops]

21.11.4.1 Multi-concept operations

[reflect.ops.over]

Modify [reflect.ops.over] as follows:



These meta-object operations are applicable to ~~to both~~ more than one of the concepts defined in the namespace.

21.11.4.4 Named operations

[reflect.ops.named]

Modify the relevant part of [reflect.ops.over] as follows:

- ...
 - for reflecting a class data member, its unqualified name;
 - for reflecting a function, its unqualified name;
 - for reflecting a specialization of a template function, its *template-name*;
 - for reflecting a function parameter, its unqualified name;
 - for reflecting a constructor, the *injected-class-name* of its class;
 - for reflecting a destructor, the *injected-class-name* of its class, prefixed by the character '~';
 - for reflecting an operator function, the *operator* element of the relevant *operator-function-id*;
 - for reflecting an conversion function, the same characters as , with reflecting the type represented by the *conversion-type-id*.
- In all other cases (for instance for reflecting a lambda object), the string's value is the empty string for `get_name<T>` and implementation-defined for `get_display_name<T>`.

21.11.4.7 Member operations

[reflect.ops.member]

Modify the relevant part of [reflect.ops.over] as follows:

With *ST* being the scope of the declaration of the entity ~~or~~, typedef or value reflected by , *S* is found as the innermost scope enclosing *ST* that is either a namespace scope (including global scope), class scope, ~~or~~ enumeration scope, function scope, or closure type.

21.11.4.8 Record operations

[reflect.ops.record]

Modify the relevant part of [reflect.ops.record] as follows:

All specializations of these templates shall meet the _____ requirements ([meta.rqmts]).
 The nested type named _____ is an alias to an _____ specialized with _____ types that
 reflect the following subset of **data**-members of the class reflected by _____ :

- for _____, all data **(function)** members.
- for _____, all public data **(function)** members;
- for _____, all data **(function)** members that are accessible from the scope of the invocation of _____ which (directly or indirectly) generated _____.

The order of the elements in the _____ is the order of the declaration of the **data** members in the class reflected by _____.

Remarks: The program is ill-formed if _____ reflects a closure type.

All specializations of these templates shall meet the _____ requirements ([meta.rqmts]).
 The nested type named _____ is an alias to an _____ specialized with _____ types that
 reflect the following subset of function members of the class reflected by _____ :

- for _____, all constructors.
- for _____, all destructors;
- for _____, all conversion functions [class.conv.fct] and operator functions [over.oper].

The order of the elements in the _____ is the order of the declaration of the members in the class reflected by _____.

Remarks: The program is ill-formed if _____ reflects a closure type.

Modify the relevant part of [reflect.ops.record] as follows:

 All specializations of _____ shall meet the _____ requirements ([meta.rqmts]). If
 reflects a class that is marked with the *class-virt-specifier*, the base characteristic of the respective
 template specialization is _____, otherwise it is _____.

21.11.4.10 Value operations **[reflect.ops.value]**

Modify the relevant part of [reflect.ops.value] as follows:

 All specializations of _____ shall meet the _____ requirements ([meta.rqmts]). If
 reflects a variable declared with the *decl-specifier*, the base characteristic of the respective
 template specialization is _____, otherwise it is _____.

- and v

[reflect.ops.derived]

[reflect.ops.namespace]

mee

1

```
[reflect.ops.fctparam]
```

```
[reflect.ops.callable]
```

All specializations of this template shall meet the requirements ([meta.rqmts]). The nested type named `__reflected_type` is an alias to an `__reflected_type` specialized with `__reflected_type` types that reflect the parameters of the function reflected by `__reflected_type`. If that function's *parameter-declaration-clause* [dcl.fct] terminates with an ellipsis, the `__reflected_type` contains an additional, final element of type `__reflected_type` for which `__reflected_type` is `__reflected_type`.

All specializations of these templates shall meet the requirements ([meta.rqmts]). If their template parameter reflects an entity that is (where applicable implicitly or explicitly) declared as `__reflected_type` (for `__reflected_type`), `__reflected_type` (for `__reflected_type`), as an inline function [dcl.inline] (for `__reflected_type`), or as deleted (for `__reflected_type`), the base characteristic of the respective template specialization is `__reflected_type`, otherwise it is `__reflected_type`.

21.11.4.15 FunctionCallExpression operations [reflect.ops.fctcall]

All specializations of `__reflected_type` shall meet the requirements ([meta.rqmts]). The nested type named `__reflected_type` is the `__reflected_type` type reflecting the function or callable object invoked by the function call expression which is reflected by `__reflected_type`.

21.11.4.16 Function operations [reflect.ops.fct]

All specializations of `__reflected_type` shall meet the requirements ([meta.rqmts]), with a static data member named `__reflected_type` of type `__reflected_type` and value `__reflected_type`, where

- for non-static member functions, `__reflected_type` is the pointer to member function type of the member function reflected by `__reflected_type` and `__reflected_type` a pointer to the member function; otherwise,
- `__reflected_type` is `__reflected_type`, where `__reflected_type` is the type of the function reflected by `__reflected_type` and `__reflected_type` is the address of that function.

21.11.4.17 RecordMemberFunction operations [reflect.ops.memfct]

All specializations of these templates shall meet the requirements ([meta.rqmts]). If their template parameter reflects a member function that is (for), (for), (for), declared with a *ref-qualifier* (for) or (for), implicitly or explicitly (for), pure virtual (for), or marked with (for) or (for), the base characteristic of the respective template specialization is , otherwise it is .

21.11.4.18 SpecialMemberFunction operations [reflect.ops.specialfct]

All specializations of these templates shall meet the requirements ([meta.rqmts]). If their template parameter reflects a special member function that is implicitly declared (for) or that is defaulted (for), the base characteristic of the respective template specialization is , otherwise it is .

21.11.4.19 Constructor operations [reflect.ops.ctor]

All specializations of this template shall meet the requirements ([meta.rqmts]). If the template parameter reflects an explicit constructor, the base characteristic of the respective template specialization is , otherwise it is .

21.11.4.20 Destructor operations [reflect.ops.dtor]

All specializations of these templates shall meet the requirements ([meta.rqmts]). If the template parameter reflects a virtual (for) or pure virtual (for) destructor, the base characteristic of the respective template specialization is , otherwise it is .

21.11.4.21 ConversionOperator operations [reflect.ops.convfct]

All specializations of this template shall meet the requirements ([meta.rqmts]). If the template parameter reflects an explicit conversion function, the base characteristic of the respective template specialization is , otherwise it is .

21.11.4.22 Lambda operations [reflect.ops.lambda]

All specializations of this template shall meet the `[[meta.rqmts]]` requirements. The nested type named `[[lambda-capture]]` is an alias to an `[[lambda-capture]]` specialized with `[[lambda-capture]]` types that reflect the captures of the closure object reflected by `[[lambda-capture]]`. The elements are in order of appearance in the *lambda-capture*.

All specializations of these templates shall meet the `[[meta.rqmts]]` requirements. If the template parameter reflects a closure object with a *capture-default* that is `[[capture-default]]` (for `[[capture-default]]`) or `[[capture-default]]` (for `[[capture-default]]`), the base characteristic of the respective template specialization is `[[capture-default]]`, otherwise it is `[[capture-default]]`.

All specializations of this template shall meet the `[[meta.rqmts]]` requirements. If the template parameter reflects a closure object with a `[[call-operator]]` call operator, the base characteristic of the respective template specialization is `[[call-operator]]`, otherwise it is `[[call-operator]]`.

21.11.4.23 LambdaCapture operations [reflect.ops.lambdacapture]

All specializations of this template shall meet the `[[meta.rqmts]]` requirements. If the template parameter reflects an explicitly captured entity, the base characteristic of the respective template specialization is `[[explicitly-captured]]`, otherwise it is `[[explicitly-captured]]`.

All specializations of this template shall meet the `[[meta.rqmts]]` requirements. If the template parameter reflects an *init-capture*, the base characteristic of the respective template specialization is `[[init-capture]]`, otherwise it is `[[init-capture]]`.

Concepts and Operations

Following P0194's lead, function reflection requires a couple new concepts to restrict operations on the meta level. It builds upon the concepts introduced by P0194 and, like P0194, all declarations are inside the `[[namespace]]` namespace. Some of the concepts below extend P0194's concepts. Whenever a concept A requires another concept B, the operations defined for concept B are also available for concept A, building a graph similar to an inheritance graph.

Proper wording will be provided when needed (and when P0194 has progressed through LWG, increasing to the authors' experience on wording of reflection papers); we believe that missing wording should not hinder SG7's design discussion. The wording is planned to be similar to that of P0194; specifically, it uses nested `[[concept]]` and `[[entity]]` entities, with the usual using declarations `[[using-declaration]]` and `[[using-declaration]]`.

Extending operands of

P0194 allows certain types and variables as operands of `[[operator]]`. This paper extends this set:

- invoking `[[operator]]` on a function call expression generates a `[[operator]]`;
- invoking `[[operator]]` on a closure object generates a `[[operator]]`;
- invoking `[[operator]]` on a parameter name generates a `[[operator]]`;
- invoking `[[operator]]` on a lambda capture generates a `[[operator]]`;

The way to obtain a `constexpr` might seem complicated. But instead of inventing a new syntax, for instance `constexpr auto add = ...`, re-using the mechanism of matching the address of an overloaded function name `&add`, or syntax similar to function definition `constexpr auto add = ...`, we rely on the existing, familiar rules for function overload resolution and generate the `constexpr` only when a concrete overload is selected. This approach also works for constructors and operators, for instance `constexpr auto add = ...` reflects the call of the default constructor of `int`; `constexpr auto add = ...` reflects the call of the built-in addition operator for type `int`.

Requires `constexpr` and `constexpr`. Represents a function parameter. Unlike `constexpr`, it does not offer a `constexpr` interface. The name of a parameter is one of the names used in an unspecified declaration. If at least one of the declarations does not specify the parameter name, then `constexpr` is allowed to return an empty string. Its scope is the `constexpr` declaring this `constexpr`.

Given the lack of relevance that C++ attributes to parameter names in function declarations one might ask: "o really?!" We believe that the answer is "yes": parameter names often carry significant meaning. Examples:

The bare combination of type and index is almost meaningless in these cases. There are many reflection applications that can benefit from this, for instance:

- remote procedure calls identifying parameters by name;
- code exposing C++ functions to scripting frameworks;
- GUI generation, converting parameter names to text box labels;
- documentation generation, referencing function parameters in a meaningful way.

To put it differently: functions without parameter names are like classes without member names. And `constexpr` is not a replacement for classes.

Another common objection is that multiple declarations could potentially have different parameter names ascribed. This concern is mitigated in two ways:

1. Modern coding conventions have the declarations for a particular function showing up in exactly one header file.
2. Modern coding conventions discourage the use of different argument names between function declarations (in a header) and function definitions (in a '.cpp' file). Dedicated compiler warnings exist to protect against this case.

Operations

Indicates that a `constexpr` reflects an ellipsis.

Indicates that a `constexpr` has a default value.

Requires `std::function`, `std::weak_ptr` and `std::shared_ptr`. Represents a function or lambda, including operators, constructors and destructors - i.e. anything this paper is dealing with.

Operations

Returns an `std::vector` of `std::string`s of the reflected `std::function`.

Returns whether the function was declared as `constexpr` or `consteval`, respectively.

Returns whether the function is an inline function. With `std::is_inline_v` is `true` for both `constexpr` and `consteval`.

Returns whether the function was defined as `constexpr` before the invocation of `std::function`.

Requires `std::function`. Reflects a call of a concrete function or other callable.

Operations

Returns the `std::function` that was invoked in a `std::function`.

Requires `std::function` and `std::weak_ptr`. Represents a function or lambda, excluding constructors and destructors.

Operations

Returns a pointer to the function. This is a pointer-to-member for non-static member functions (including lambda calls), and a function pointer otherwise. It is ill-formed to invoke this for deleted functions. Example: `decltype(auto) f = &f;` holds the address of `f`.

Requires `std::is_member_function_pointer` and `std::is_member_function_pointer`. Represents a member function, excluding constructors and destructors.

Operations

Returns whether this is a static member function.

Returns whether the function is declared as `constexpr` or `constexpr`, respectively.

Returns whether the function is declared as with a *ref-qualifier* being `&` or `&&`, respectively.

Returns whether the function is a virtual or pure virtual function, respectively. For

the value of `std::is_member_function_pointer` is `true`, irrespectively of whether `std::is_member_function_pointer` is implicit or explicit.

Returns whether the function is declared as `constexpr` or `consteval`, respectively.

Requires `constexpr`. Represents a special member function.

Operations

Returns whether the special member function is known to be implicitly declared at the point of invocation of `is_special_member_function`.

Returns whether the function is defined as `constexpr` before the invocation of `is_special_member_function`, independently of whether the special member function is implicitly or explicitly declared.

Requires `constexpr` and `constexpr`. Represents a constructor. The base name of the constructor is the base name of the constructor's class. Even though the standard explicitly says that constructors do not have a name, for usability purposes (e.g. generating messages), having them state the class name is a usability improvement. Some instances of `is_constructor` might also satisfy `is_constructor`.

Operations

Returns whether the constructor is known to be declared as `constexpr`.

Requires `constexpr`, `constexpr` and `constexpr`. Represents a destructor. The base name is the base name of the destructor's class, prefixed with `~`.

Returns whether the destructor is a virtual or pure virtual function, respectively. For

the value of `isImplicit` is `true`, irrespective of whether `isImplicit` is implicit or explicit.

Requires `isImplicit`. Some instances might implement `isImplicit` or `isImplicit`. Represents an operator. The base name is the operator "symbol", for instance `+`.

Requires `isImplicit`. Represents a conversion operator. The base name is the base name of the operator's target type, for instance `int`.

Operations

Returns whether the function is declared as `inline`.

Requires `isImplicit`. Represents a closure type, excluding those for generic lambdas. Its base name is the empty string.

Operations

Returns an `int` of `int` s.

Returns whether the *capture-default* is `true`.

Returns whether the *capture-default* is `true`.

Returns `bool` if the lambda was declared as `mutable`, `const` otherwise.

Requires `std::lambda::capture`. Represents a lambda capture as introduced by the capture list or by capture defaults. The `std::lambda::capture`'s scope is its `std::lambda::capture`.

Operations

Returns whether the entity was captured explicitly.

Returns whether the entity is an *init-capture*.

Extending

This proposal adds the following interfaces to the `std::lambda` concept of P0194:

Returns an `std::vector<std::lambda::capture>` of `std::lambda::capture`s representing a class's public member functions (for `std::lambda::capture::public`), member functions accessible from the point of invocation of `std::lambda::capture::public` (for `std::lambda::capture::public`) and all member functions, irrespective of their accessibility (for `std::lambda::capture::all`). This includes *static* member functions, but not *friend* functions.

Returns an `std::vector<std::lambda::capture>` of a class's `std::lambda::capture`s, `std::lambda::capture`s and `std::lambda::capture`s, respectively.

Acknowledgments

Thanks to Jackie Kay who provided valuable feedback, criticism and suggestions!

References

1. *Static reflection. Rationale, design and evolution.* [p0385](#)
2. *Static reflection in a nutshell.* [p0578](#)